Take-Home Full-Stack Assessment

Unified Inbox

You are building a platform that searches across someone's Gmail, Slack, and the web from one place, and lets them compose replies with a deliberate confirmation step before anything is ever sent. A user connects accounts once, runs a single query that fans out across every source, sees one merged result list, and can act on a result by drafting a reply that only goes out after they explicitly confirm exactly what will be sent and where.

The provider adapter layer and the safe-send gate are the centerpiece. Each connected source is a pluggable adapter that runs as a background worker and returns results in one common shape, so new sources drop in without touching the merge layer. Anything that sends or posts passes through a draft, review, confirm gate that is safe to retry without ever double-sending. Think of the adapters and the send gate as the reusable core; the unified inbox is one product built on top of them.

This is a proof of concept, not a production system. We have intentionally left many decisions to you, and how you make them is part of what we are evaluating.

Time expectation: ~5-6 days with effective use of AI coding tools.

Recommended Stack

Layer	Recommendation
Frontend	React or any React-based framework (Next.js, Remix, etc.)
Backend	Node.js or Python, pick one and keep the adapters and gate in it
Database	Any persistent storage (PostgreSQL, MongoDB, Firebase, etc.)
Infrastructure	Google Cloud preferred, but equivalent architecture is acceptable

---

What the system does

Connections

A user connects one or more Gmail accounts and one or more Slack workspaces. Each connection is a separate OAuth grant, scoped narrowly rather than to blanket access, reusable across searches. Tokens are stored securely and refreshed silently. Connections show status (active / expired / errored) and have a one-click reconnect that preserves identity so dependent state is not disrupted. Web search needs no connection. Use throwaway or test accounts for development; never require a candidate or reviewer to connect a real personal account.

Unified search

One query fans out to every connected adapter concurrently. Results return in a single common shape and merge into one ranked list. A slow source must not block a fast one: partial results stream in as each adapter returns, and the UI shows which sources are still working.

Provider adapters

Each source is an adapter behind a common interface: given a query, return normalized results. Each runs as an independent background worker, so the merge layer never knows or cares which sources exist. Adding a new source is writing one adapter, nothing else. Implement real adapters for Gmail and Slack and a web-search adapter (a real search API or a clearly-labeled mock). Shapes are fixed:

```
interface SearchAdapter {
  source: "gmail" | "slack" | "web";
  search(query: string, ctx: AdapterContext): Promise<Result[]>;
}

interface Result {
  source: string;
  id: string;
  title: string;
  snippet: string;
  author?: string;
  timestamp?: string;   // ISO 8601
  url: string;
}
```

Safe send

Composing a reply to an email or a Slack message produces a draft. Sending it requires an explicit confirmation that shows exactly what will be sent, to whom, and on which source. This is not a single misfireable click: an external message going out names the recipient or channel and requires deliberate confirmation. A send must be idempotent: retrying after a network blip or a double-tap must never send twice. Use an idempotency key carried on the draft.

```
interface Draft {
  id: string;
  channel: "gmail" | "slack";
  to: string;              // recipient address or channel
  subject?: string;        // email only
  body: string;
  idempotency_key: string;
}
// POST /drafts           -> creates a draft, returns it
// POST /drafts/{id}/send -> sends; safe to retry, returns the same result for the same
key
```

Failure handling

Transient failures (a flaky network call, a rate limit) are auto-retried with backoff. Permanent failures (auth revoked, recipient invalid, quota exhausted) are surfaced with a distinct, honest reason and not auto-retried; the operator decides when to retry. Detect a revoked or expired grant specifically and route the user to reconnect rather than failing opaquely.

Operations and history

Every search and every send is recorded with its status. Per-item detail shows what was sent, to whom, the delivery outcome, attempt count, and the full error on failure. Operators can retry a failed send and re-run a search.

API access

Beyond the UI, expose a small read-and-write REST API so an external script or agent can run a search, list results, create a draft, confirm a send, and retry a failed one. Authenticate with a per-user API key scoped to that user's data. Document it in the README.

Success criteria

These are concrete checks your submission should pass, written as input plus expected observable outcome. Good submissions turn the testable ones into automated tests; your reviewer walks the full list against your deployed app. Use test or throwaway accounts throughout, and a known test recipient you can inspect.

1. **Concurrent fan-out with partial results.** With Gmail, Slack, and web all connected, run a query where one source is artificially slow. The fast sources' results appear first while the UI marks the slow source as still working; once it returns, the merged list contains results from all three. The slow source never blocks the fast ones.

2. **Normalization.** Inspect the `GET` search response from the API. Every result, whatever its source, has `source`, `id`, `title`, `snippet`, and `url` populated and conforms to the common `Result` shape. A consumer can render the list without knowing which source each item came from.

3. **Idempotent send.** Create a draft to your test recipient carrying idempotency key `K`. Call `POST /drafts/{id}/send` twice with key `K` (simulating a double-tap or a retried request). Exactly one message arrives at the recipient; the second call returns the same result as the first, not a second delivery. This is the check we verify most carefully.

4. **Confirm friction.** There is no path, in the UI or the API, that sends an external message in a single misfireable step. Sending requires a confirmed draft whose recipient or channel and exact body were shown first. A raw one-shot "send this text to this person" with no confirm is rejected.

5. **Revoked grant becomes a reconnect.** Invalidate a connection's token, then run a search or send against that source. The result is a distinct "needs reconnect" state, not a generic failure, and one-click reconnect restores it while preserving identity so dependent state is intact. After reconnect, the same operation succeeds.

6. **Transient versus permanent failure.** A simulated transient error (a 503 or rate limit) on a send is retried with backoff and resolves or is surfaced as transient. A permanent error (invalid recipient, revoked auth) is surfaced immediately with a distinct reason and is not auto-retried; the operator decides.

7. **History fidelity.** Every search and send appears in history with the correct status. A failed send shows its attempt count and full error message and can be retried by the operator from the detail view.

Quality bar

The adapter layer and the send gate must be a standalone module behind the documented API, and the UI must be a pure consumer of it. We should be able to run a search and send a message entirely through the API with no UI present.

The entire interface must be mobile-friendly across every surface: connections, search, compose, confirm, and history.

Automated tests are required, and the idempotent send is where we look hardest. Prove that issuing the same send twice (same idempotency key) results in exactly one delivery; that a slow adapter does not block fast ones; that results normalize into the common shape; and that a revoked grant surfaces as a reconnect prompt rather than a generic error.

Real seed data: synthetic searches and sends in every status so history and detail views can be demonstrated meaningfully. Clearly distinguish seed data from real processing.

Thoughtfulness over completeness. A submission that makes sending genuinely safe and searching genuinely unified, and treats revoked grants and double-sends with care, beats a feature-complete one that is brittle.

Deliverables

- **GitHub repository** with a clear README covering architecture, local setup, OAuth setup, the web-search choice, seeding instructions, and how to run the tests. The project must run in GitHub Codespaces.
- **Deployed URL** with real OAuth flows for Gmail and Slack working end to end.
- **Recorded demo (15-30 minutes)** in two parts. First, the working system: connection management, a unified search across all sources, composing a reply and the confirm-before-send step, a demonstration that a retried send does not double-send, a reconnect after an invalidated grant, and at least one failure scenario. Second, a detailed architectural walkthrough: talk us through your multi-account OAuth design (token storage, refresh, recovery from invalidated grants), the adapter design, the idempotent-send strategy, the key tradeoffs you weighed, and what you would change with more time. We want to hear your reasoning, not just see the result.

Evaluation

We weigh:

- **Adapter layer**: clean common interface, correct normalization, concurrent fan-out, partial results, easy to add a source
- **Safe-send gate**: confirmation friction that prevents misfires, idempotency that prevents double-sends
- **Multi-account OAuth**: secure token storage and silent refresh, recovery from invalidated grants
- **Search ergonomics**: fast, legible, honest about which sources are still working
- **Operations clarity**: observable history, recoverability from common failures
- **Clean module boundary**: adapters and gate standalone, UI a pure consumer
- **Mobile usability** across every surface
- **End-to-end reliability**: working deployment with real OAuth, no silent failures

The best submissions show real polish and judgment in every detail, especially around sending safely and recovering from broken grants.

A quick note on the assessment before you begin. Participation is voluntary and unpaid. It is a screening exercise only, not work for Genius Innovation Lab or for any client, and we do not commercialize your submission or use it in any product or client deliverable. The only exception is that, with your permission, we may share your submission with a prospective client solely so they can evaluate your fit for a role, and never for their own use. You retain full ownership of everything you create and are free to publish it, post it on GitHub, or include it in your portfolio however you wish. Completing the assessment does not create an employment, contractor, or agency relationship, carries no entitlement to compensation, and does not guarantee an offer or placement. You are free to decline or stop at any time by simply replying to let us know.